Reviewer Pack — Minimal Rank of Twisted Alexander Module over XOR-AND Tree W…

Entry cb84f01b5db0 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-26 19:42:33 UTC

Minimal Rank of Twisted Alexander Module over XOR-AND Tree Width

Entry ID: `cb84f01b5db0` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-26 19:42:33 UTC

1. Conjecture

Field A (mathematical branch): Twisted Algebraic Topology (Alexander Modules) **Field B** (complexity object): Complexity Theory: XOR-AND Tree Width

Statement:

> {'text': 'For any instance I of an XOR-AND tree with width w, the minimal rank of the twisted Alexander module A_I associated with I is bounded by O(w^2). Specifically, there exists a constant c > 0 such that ρ(A_I) ≤ cw^2 for all I.', 'counterexample': 'Provide an XOR-AND tree T with width w and a corresponding twisted Alexander module A_T satisfying ρ(A_T) > cw^2.'}

Rationale (proposer's reasoning):

> {'text': 'Twisted Alexander modules are known to capture topological information about knots and links, which may provide insights into the structure of XOR-AND trees. The proposed upper bound on the minimal rank could lead to new algorithms or lower bounds for XOR-AND tree width.'}

Taxonomy category: `Alexander_Module_XOR_AND_Tree` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `4b970b07197f229e`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> For an XOR-AND tree with width w, if the minimal rank of the twisted Alexander module ρ(A_I) is less than or equal to cw^2 for all generated trees and at least 80% of seeds, then the conjecture is supported. Falsification occurs if any seed produces a tree where ρ(A_T) > cw^2.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM_1	LLM_2
RELATIVIZATION	UNCERTAIN	1.00	HITS	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 9 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - twisted Alexander modules AND XOR-AND tree width - minimal rank of Alexander modules in terms of XOR-AND tree width - Alexander modules and complexity theory with focus on XOR-AND tree width

Top relevant hits considered: - [http://arxiv.org/abs/2605.13288v1] Twisted Alexander polynomials of a knot for group extensions - [http://arxiv.org/abs/1107.3283v3] The twisted Alexander polynomial for finite abelian covers over three manifolds with boundary - [http://arxiv.org/abs/0706.2499v1] Alexander polynomials: Essential variables and multiplicities - [http://arxiv.org/abs/2309.05856v1] Multi-Point Detection of the Powerful Gamma Ray Burst GRB221009A Propagation through the Heliosphere on October 9, 2022 - [http://arxiv.org/abs/2501.11667v1] A One-Dimensional Energy Balance Model Parameterization for the Formation of CO2 Ice on the Surfaces of Eccentric Extras - [http://arxiv.org/abs/2106.09310v1] Trends and Characteristics of High-Frequency Type II Bursts Detected by CALLISTO Spectrometers - [http://arxiv.org/abs/math/9806161v1] Chern Classes in Alexander-Spanier Cohomology - [http://arxiv.org/abs/hep-ex/0108011v1] Cerenkov Particle Identification in FOCUS

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
import math
from fractions import Fraction

def gaussian_elimination(matrix):
    rows, cols = len(matrix), len(matrix[0])
    for i in range(rows):
        # Find pivot
        max_row = i
        for j in range(i+1, rows):
            if abs(matrix[j][i]) > abs(matrix[max_row][i]):
                max_row = j
        matrix[i], matrix[max_row] = matrix[max_row], matrix[i]

        # Eliminate below pivot
        for j in range(i+1, rows):
            factor = Fraction(matrix[j][i], matrix[i][i])
            for k in range(cols):
```

```
                matrix[j][k] -= factor * matrix[i][k]

    return matrix

def compute_rank(matrix):
    matrix = [row[:] for row in matrix]
    gaussian_elimination(matrix)
    rank = 0
    for row in matrix:
        if any(row):
            rank += 1
    return rank

def generate_xor_and_tree(depth, max_children=2):
    if depth == 0:
        return random.choice([True, False])
    children = [generate_xor_and_tree(depth-1) for _ in range(random.randint(1, max_children))]
    return (children[0], children[1])

def compute_twisted_alexander_module(tree):
    if isinstance(tree, bool):
        return [[Fraction(1)]]

    left_module = compute_twisted_alexander_module(tree[0])
    right_module = compute_twisted_alexander_module(tree[1])

    # Concatenate modules
    new_module = []
    for row_left in left_module:
        for row_right in right_module:
            newRow = [row_left[i] + row_right[i] for i in range(len(row_left))]
            new_module.append(newRow)

    return new_module

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n_values = [5, 10, 15, 20, 30, 40]
    results = []

    for n in n_values:
        tree = generate_xor_and_tree(n)
        module = compute_twisted_alexander_module(tree)
        rank = compute_rank(module)
        results.append((n, rank))

    total_rank = sum(rank for _, rank in results)
    mean_rank = total_rank / len(results)
    conjecture_holds = all(rank <= 2 * n**2 for _, rank in results) and (len([r for r, _ in result

    return {
        "metric_name": "minimal_rank",
        "metric_value": mean_rank,
        "instances_tested": len(results),
        "conjecture_holds": conjecture_holds,
        "counterexample": "" if conjecture_holds else f"rank={max(rank for _, rank in results)}, e
```

```
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [random.randint(1, 1000) for _ in range(30)]

    results = []
    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result)

    mean_value = sum(result["metric_value"] for result in results) / len(results)
    support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

    if support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_value} std={math.sqrt(sum((result['metric_value'] - m
    elif any(not result["conjecture_holds"] for result in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conj
        print(f"RESULT: FALSIFIED counterexample=\"rank exceeds bound\" first_failing_seed={first_
    else:
        print("RESULT: INCONCLUSIVE reason=insufficient_evidence")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_29500f30.py", line 96, in <module>
    trial_result = run_trial(seed)
                   ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_29500f30.py", line 73, in run_trial
    tree = generate_xor_and_tree(n)
           ^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_29500f30.py", line 48, in generate_xor_
   children = [generate_xor_and_tree(depth-1) for _ in range(random.randint(1, max_children))]
                ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_29500f30.py", line 48, in generate_xor_
   children = [generate_xor_and_tree(depth-1) for _ in range(random.randint(1, max_children))]
                ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_29500f30.py", line 48, in generate_xor_
   children = [generate_xor_and_tree(depth-1) for _ in range(random.randint(1, max_children))]
                ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  [Previous line repeated 1 more time]
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_29500f30.py", line 49, in generate_xor_
    return (children[0], children[1])
                         ~~~~~~~~^^^
IndexError: list index out of range
```

8. Critic adversarial review

Critic verdict: `CHALLENGE`

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: `INCONCLUSIVE`

Reasoning:

The test code crashed before producing data, which means it did not complete its execution to verify the conjecture. | next: Re-run the test with increased robustness checks and error handling to ensure that it can handle edge cases without crashing.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	10997
2	preregistration	ollama_remote	glm4:latest	0	0	5657
3	novelty	ollama_remote	glm4:latest	0	0	4562
4	novelty	ollama_remote	glm4:latest	0	0	6659
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	25095
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8105
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8320
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10928
9	judge	ollama_remote	glm4:latest	0	0	10441

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 90763 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/cb84f01b5db0.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/cb84f01b5db0.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/cb84f01b5db0.tar.gz` (if generated)